

Fiche Résumé : Design Pattern Composite

1. Présentation Générale & Intention Métier

- **Nom du Pattern** : Composite
- **Catégorie** : Structurel
- **Intention Métier** : Permet de composer des objets dans des structures en arbre pour représenter des hiérarchies composant/composé. Le pattern Composite permet aux clients de traiter les objets individuels (feuilles) et les compositions d'objets (composites) sous la forme d'un arbre.
- **Problématique Générale** : Traiter des structures arborescentes imbriquées sans obliger le code client à effectuer des vérifications de types répétitives ou à gérer plusieurs collections distinctes.

2. Diagramme de Classes Générique & Solution Officielle

Structure du pattern Composite :

- **Component (Interface / Classe Abstraite)** : Définit l'interface commune pour tous les objets de la hiérarchie (méthodes métier comme `operation()`, et la gestion des enfants comme `add()`, `remove()`).
- **Leaf (Feuille)** : Représente les objets n'ayant pas d'enfants. Implémente le comportement de base.
- **Composite (Conteneur)** : Représente un composant contenant des enfants (`List<Component>`). Implémente les méthodes de l'interface en déléguant le travail à ses enfants.
- **Client** : Manipule l'ensemble des objets à travers l'interface commune `Component`.

3. Tableau des Classes Participantes

Nom Générique (GoF)	Description / Rôle dans la structure
Component	Interface ou classe abstraite définissant l'interface commune pour tous les objets simples et conteneurs.

Nom Générique (GoF)	Description / Rôle dans la structure
Leaf	Objet élémentaire sans enfant qui implémente le comportement métier de base.
Composite	Conteneur stockant une collection d'objets Component et déléguant les opérations récursivement à ses enfants.
Client	Manipule tous les éléments de la structure arborescente de manière uniforme via l'abstraction Component .

4. Exemple Concret (Calcul du Poids de Colis)

- **Problématique Métier** : Une entreprise de logistique doit calculer le poids total de colis. Un colis (**Carton**) peut contenir des articles simples (**Produit**) ou d'autres **Carton** emboîtés.
- **Modélisation Imparfaite** : **Carton** possède deux listes distinctes (**List<Produit>** et **List<Carton>**), obligeant à parcourir manuellement et récursivement chaque type.
- **Solution avec le Pattern Composite** :
 - **Classe Abstraite Element** : Déclare la méthode **getPoids() : int**.
 - **Classe Produit (Leaf)** : Implémente **Element**, retourne son poids direct.
 - **Classe Carton (Composite)** : Implémente **Element**, contient une unique **List<Element>** et calcule le poids global par sommation récursive des enfants.

5. Analyse de la SOLIDité du Pattern

- **Single Responsibility Principle (SRP)** : Séparation claire entre la gestion des données simples (**Leaf**) et la gestion du conteneur/parcours arborescent (**Composite**).
- **Open/Closed Principle (OCP)** : Facilité d'extension. De nouveaux types de feuilles ou de conteneurs peuvent être ajoutés sans modifier l'interface **Component** ni le code client existant.
- **Liskov Substitution Principle (LSP)** : Un objet **Composite** ou **Leaf** peut être interchangé à tout endroit où l'abstraction **Component** est attendue.

- **Interface Segregation Principle (ISP)** : L'interface **Component** reste focalisée sur les fonctionnalités métier partagées.
- **Dependency Inversion Principle (DIP)** : Le client et le conteneur dépendent uniquement de l'abstraction **Component**, jamais des classes concrètes.

6. Limites et Comparaison avec d'autres Patterns

- **Limites** :
 - Cela peut rendre le design trop général si certaines feuilles ne doivent pas autoriser certaines opérations.
 - La gestion fine des opérations d'ajout/suppression dans l'interface parente peut parfois poser un compromis entre transparence et sécurité.
- **Comparaison avec le Pattern Décorateur** :
 - **Similitude** : Les deux patterns reposent sur une structure récursive.
 - **Différence** : Le **Décorateur** possède un seul composant enfant et ajoute des responsabilités ou des comportements supplémentaires à l'objet enveloppé. Le **Composite** contient une collection variable d'enfants et se contente d'agréger ou de sommer les résultats de son arborescence.